

Reviewer Pack — Minimal Rank of Quiver Representations over Frege Proof Tree...

Entry ae94247c6126 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 23:00:30 UTC

Minimal Rank of Quiver Representations over Frege Proof Trees

Entry ID: ae94247c6126 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 23:00:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quiver Representations) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

[‘For a given Frege proof tree T , the minimal rank of any quiver representation associated with the clauses in T is upper bounded by the depth of T , i.e., $E[\text{MinRank}(\text{quiver})] \leq \text{Depth}(T)$.’, ‘Additionally, for every instance of size n with a Frege proof tree of depth d , there exists a quiver representation whose dimension is less than or equal to 2^d .’, ‘Furthermore, if an instance has a Frege proof tree T of depth less than or equal to $\log(n)$, then the corresponding quiver representation can be computed in polynomial time.’]

Rationale (proposer’s reasoning):

[‘Quiver representations are a tool from algebraic combinatorics that can encode the structure of graphs and other discrete structures. By relating their ranks to Frege proof trees, we may uncover new structural insights into the complexity of computational problems.’, ‘The conjecture builds on the idea that the complexity of a proof system can be understood through the algebraic properties of its clauses. It suggests that simpler proof structures correspond to more structured algebraic representations.’, ‘This bridge could potentially reveal new methods for proving lower bounds on Frege proof length, which is a central problem in complexity theory.’]

Taxonomy category: quiver_representation_froge_proof (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8aaf17690d42c277

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Frege proof tree T with size $n \leq 40$ and depth ranging from 1 to $\log(n)$, if the minimal rank of any associated quiver representation is less than or equal to $\text{Depth}(T)$ AND there exists a quiver representation with dimension $\leq 2^{\text{Depth}(T)}$, then the conjecture

is supported. If for any instance, the minimal rank exceeds $\text{Depth}(T)$ OR no quiver representation has dimension $\leq 2^{\text{Depth}(T)}$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quiver representations" AND "Frege proof complexity" - "minimal rank" AND "quiver representations" AND "proof tree" - "polynomial time computation" AND "quiver representation" AND "Frege proof tree depth"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_frege_tree(depth, max_depth):
        if depth == max_depth:
            return []
        else:
            return [generate_frege_tree(random.randint(1, depth), depth - 1)]

    def compute_min_rank(tree):
        if not tree:
            return 0
        left_rank = compute_min_rank(tree[0])
        right_rank = compute_min_rank(tree[1]) if len(tree) > 1 else 0
        return max(left_rank, right_rank) + 1

    def generate_quiver_representation(tree):
        if not tree:
            return 1
        left_rep = generate_quiver_representation(tree[0])
        right_rep = generate_quiver_representation(tree[1]) if len(tree) > 1 else 1
        return left_rep * right_rep
```

```

n = random.randint(5, 40)
depth = math.ceil(math.log(n))
tree = generate_frege_tree(depth, depth)

min_rank = compute_min_rank(tree)
quiver_dimension = generate_quiver_representation(tree)

conjecture_holds = (min_rank <= depth) and (quiver_dimension <= 2 ** depth)
counterexample = "" if conjecture_holds else f"Depth={depth}, MinRank={min_rank}, QuiverDim={quiver_dimension}"

return {
    "metric_name": "MinRank",
    "metric_value": min_rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) # Default seeds

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support={support_fraction:.4f}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}

```

```

TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'MinRank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
RESULT: SUPPORTED mean=0.0000 std=0.0000 support_fraction=1.0000

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only one instance tested, which is insufficient to draw a conclusion about the conjecture’s validity. This is a clear case of ‘n too small’, as only $n \leq 15$ instances were tested. The metric does not scale trivially with n, and without testing more instances, we cannot be sure that the observed behavior holds for larger inputs.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test has only one instance tested, which is insufficient to draw a conclusion about the conjecture’s validity. The critic challenged the result due to the small number of instances tested and the lack of scalability with respect to input size. | next: Test more instances with varying sizes and depths to validate the conjecture across a broader range of inputs.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12349
2	preregistration	ollama_remote	glm4:latest	0	0	6387
3	novelty	ollama_remote	glm4:latest	0	0	4759
4	novelty	ollama_remote	glm4:latest	0	0	5623
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13950
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9343
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7071
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8670
9	critic	ollama_remote	glm4:latest	0	0	15043
10	judge	ollama_remote	glm4:latest	0	0	5642

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 88839 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in <research/audit/ae94247c6126.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ae94247c6126.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ae94247c6126.tar.gz` (if generated)